

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



Searching

Oleh:

Rahma Syarita Jaya NIM. 2510817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya
NIM : 2510817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
LINEAR SEARCH.....	6
A. Source Code.....	6
B. Output Program	6
C. Pembahasan	7
BINARY SEARCH	8
A. Source Code.....	8
B. Output Program	8
C. Pembahasan	9
TAUTAN GIT	11

DAFTAR GAMBAR

Gambar 1. Output Algoritma Linear Search	6
Gambar 2. Visualisasi singkat Linear Search.....	7
Gambar 3. Output Algoritma Binart Search.....	8
Gambar 4. Visualisasi singkat Binary Search	9

DAFTAR TABEL

Tabel 1 Source code Linear Search	6
Tabel 2 Source code Binary Search.....	8

LINEAR SEARCH

A. Source Code

Tabel 1 Source code Linear Search

```

1 void linear_search(const std::vector<int>& data, int
  target, Metrics& m) {
2     m.reset();
3     auto start = Clock::now();
4     m.n = data.size();
5
6     for (int i = 0; i < data.size(); i++) {
7         m.comparisons++;
8         if (data[i] == target) {
9             m.found_index = i;
10            break;
11        }
12    }
13    m.time_us = std::chrono::duration<double,
  std::micro>(Clock::now() - start).count();
14 }

```

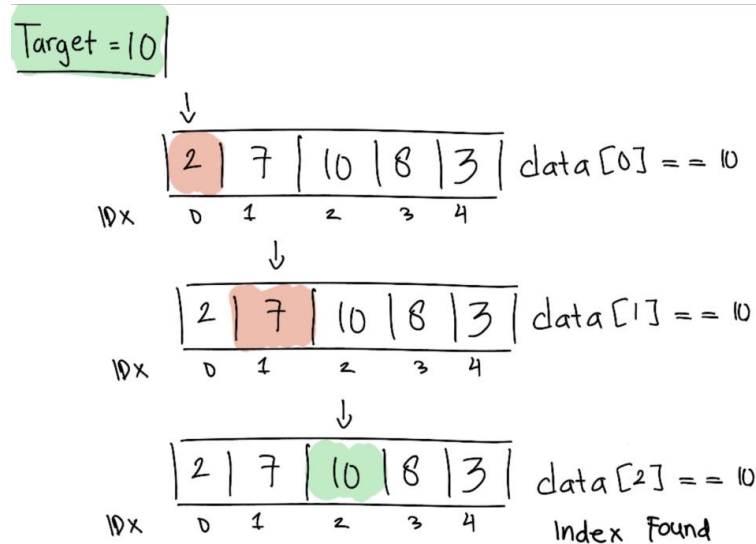
B. Output Program

>> Linear Search					
Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.100
Linear Search	Average	1000	375	374	1.700
Linear Search	Worst	1000	1000	Not Found	4.500
Linear Search	Best	10000	1	0	0.100
Linear Search	Average	10000	3746	3745	16.200
Linear Search	Worst	10000	10000	Not Found	42.700
Linear Search	Best	100000	1	0	0.000
Linear Search	Average	100000	37455	37454	180.800
Linear Search	Worst	100000	100000	Not Found	465.700
Linear Search	Best	1000000	1	0	0.200
Linear Search	Average	1000000	374541	374540	1983.600
Linear Search	Worst	1000000	1000000	Not Found	4595.500

Gambar 1. Output Algoritma Linear Search

C. Pembahasan

Pertama-tama, algoritma ini melakukan perulangan dengan indeks $i = 0$ sampai $i < \text{data.size}()$. Terdapat pengecekan suatu kondisi dengan if, apakah elemen pada indeks saat ini sama dengan target dengan $\text{data}[i] == \text{target}$. Program ini akan terus melakukan perulangan sampai kondisi terpenuhi dan data ditemukan, program menyimpan indeks tersebut ke dalam $m.\text{found_index} = i$ dan menghentikan program dengan `break` untuk menghentikan perulangan tanpa harus menunggu kondisi loop terpenuhi. Jika perulangan selesai dan target tidak ditemukan, program akan menetapkan $m.\text{found_index} = -1$.



Gambar 2. Visualisasi singkat Linear Search

Linear Search tidak membutuhkan data terurut karena algoritma ini berjalan dengan membandingkan satu persatu tiap elemen data dengan target atau bersifat iteratif karena menggunakan perulangan `for` dari indeks awal sampai akhir meskipun bisa diterapkan secara rekursif jika mau. Untuk kompleksitas waktu sendiri, Best case memiliki $\Omega(1)$ karena loop akan berjalan sekali karena data berada pada indeks paling pertama. Average dan Worst memiliki kompleksitas waktu $O(n)$. Untuk Average performa kodenya sangat bergantung pada seberapa besar ukuran data. Lalu, Worst dimana target berada di paling akhir atau not found, maka loop harus memeriksa seluruh n data sebagaimana di output comparisons bernilai sama dengan n .

BINARY SEARCH

A. Source Code

Tabel 2 Source code Binary Search

```

1 void binary_search(const std::vector<int>& data, int
  target, Metrics& m) {
2     m.reset();
3     m.n = data.size();
4     auto start = Clock::now();
5
6     int low = 0;
7     int high = data.size() - 1;
8     while (low <= high) {
9         int mid = low + (high - low) / 2;
10
11         m.comparisons++;
12         if (data[mid] == target) {
13             m.found_index = mid;
14             break;
15         }
16         if (data[mid] < target)
17             low = mid + 1;
18         else
19             high = mid - 1;
20     }
21     m.time_us = std::chrono::duration<double,
22 std::micro>(Clock::now() - start).count();
23 }

```

B. Output Program

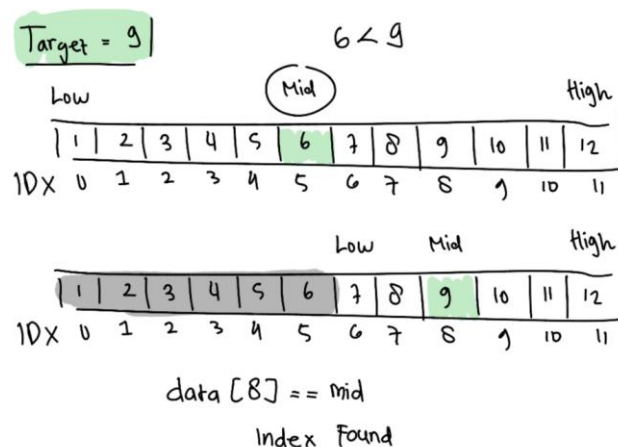
>> Binary Search					
Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.400
Binary Search	Average	1000	3	374	0.300
Binary Search	Worst	1000	10	Not Found	0.400
Binary Search	Best	10000	13	5000	0.200
Binary Search	Average	10000	13	3745	0.300
Binary Search	Worst	10000	14	Not Found	0.500
Binary Search	Best	100000	16	50000	0.200
Binary Search	Average	100000	16	37454	0.700
Binary Search	Worst	100000	17	Not Found	0.600
Binary Search	Best	1000000	19	500000	1.200
Binary Search	Average	1000000	15	374540	1.200
Binary Search	Worst	1000000	20	Not Found	1.000

Gambar 3. Output Algoritma Binart Search

C. Pembahasan

Algoritma ini bekerja dengan membagi ukuran data menjadi dua bagian secara terus-menerus hingga data yang dicari ditemukan. Pertama-tama, menetapkan $low = 0$ dan batas akhir dengan $high = data.size() - 1$. Dalam perulangan pertama dengan `while` yang berjalan selama syarat kondisi $low \leq high$ terpenuhi. Program mencari indeks nilai tengah dengan rumus $mid = low + (high - low) / 2$. Selanjutnya terdapat pengecekan dengan `if`. Dimana, `if` pertama memeriksa apakah data di tengah sama dengan target. Jika kondisi ini terpenuhi, program menyimpan indeks tersebut ke dalam `m_found_index = mid` dan langsung menghentikan fungsi dengan menghentikan perulangan, tanpa harus menunggu kondisi loop selesai..

Jika tidak sama, `if` kedua mengecek jika $data[mid] < target$, maka `low` akan diperbarui ke $mid + 1$, atau secara mudahnya data dibagi menjadi dua dan menaruh nilai `low` di bagian yang sesuai seperti `low` akan berada tetap tetapi `high` akan berubah ke $mid - 1$ ketika target lebih besar dari nilai `mid`, begitu pula sebaliknya. Lebih mudahnya, seperti gambaran berikut:



Gambar 4. Visualisasi singkat Binary Search

Binary Search perlu data yang sudah terurut karena menjadi dua bagian data berpegang dengan kurang dari atau lebih besarnya suatu target. Algoritma ini bersifat iteratif karena menggunakan perulangan `while` meskipun bisa diterapkan secara rekursif jika mau. Untuk kompleksitas waktu sendiri, Best Case memiliki $O(1)$ karena target berada tepat di nilai `mid` pertama dan hanya melakukan satu perulangan dan perbandingan sekali. Sedangkan untuk Average dan Worst memiliki kompleksitas waktu $O(\log n)$. Karena cara kerja algoritma ini membelah ruang pencarian menjadi setengah setiap iterasi. Untuk case Average, target diacak dan bisa saja ditemukan di tengah proses pembelahan, sementara Worst Case berada di ujung belahan atau not found sebagaimana dibuktikan pada output Worst-Case dimana jumlah komparasi maksimalnya bernilai mendekati dengan uji coba rumus kompleksitas.

Kompleksitas Waktu	Hasil Logaritma	Jumlah Comparassion
--------------------	-----------------	---------------------

$\text{Log}_2(1000)$	9.96578428466	9
$\text{Log}_2(10000)$	13.2877123795	13
$\text{Log}_2(100000)$	16.6096404744	16
$\text{Log}_2(1000000)$	19.9315685693	19

TAUTAN GIT

<https://github.com/DSA26-ULM/module-5-searching-jyaritta>